# PART1: Answer Questions

1. **How many OpenFlow headers with type “OFPT_FLOW_MOD” and command “OFPFC_ADD” are there among all the packets?**
2. **What are the match fields and the corresponding actions in each “OFPT_FLOW_MOD” message?**
3. **What are the Idle Timeout values for all flow rules on s1 in GUI?**

```
OpenFlow 1.4
    Version: 1.4 (0x05)
    Type: OFPT_FLOW_MOD (14)
    Length: 104
    Transaction ID: 0
    Cookie: 0x00230000dc94c6ec
    Cookie mask: 0x0000000000000000
    Table ID: 0
    Command: OFPFC_ADD (0)
    Idle timeout: 0
    Hard timeout: 0
    Priority: 10
    Buffer ID: OFP_NO_BUFFER (4294967295)
    Out port: OFPP_ANY (4294967295)
    Out group: OFPG_ANY (4294967295)
  ▸ Flags: 0x0001
    Importance: 0
  ▾ Match
      Type: OFPMT_OXM (1)
      Length: 32
    ▾ OXM field
        Class: OFPXMC_OPENFLOW_BASIC (0x8000)
        0000 000. = Field: OFPXMT_OFB_IN_PORT (0)
        .... ...0 = Has mask: False
        Length: 4
        Value: 2
    ▾ OXM field
        Class: OFPXMC_OPENFLOW_BASIC (0x8000)
        0000 011. = Field: OFPXMT_OFB_ETH_DST (3)
        .... ...0 = Has mask: False
        Length: 6
        Value: de:68:97:6c:7f:9e (de:68:97:6c:7f:9e)
    ▾ OXM field
        Class: OFPXMC_OPENFLOW_BASIC (0x8000)
        0000 100. = Field: OFPXMT_OFB_ETH_SRC (4)
        .... ...0 = Has mask: False
        Length: 6
        Value: da:cc:5d:95:16:9a (da:cc:5d:95:16:9a)
    Instruction
      Type: OFPIT_APPLY_ACTIONS (4)
      Length: 24
      Pad: 00000000
    ▾ Action
        Type: OFPAT_OUTPUT (0)
        Length: 16
        Port: 1
        Max length: 0
        Pad: 000000000000
```

```
OpenFlow 1.4
    Version: 1.4 (0x05)
    Type: OFPT_FLOW_MOD (14)
    Length: 96
    Transaction ID: 33
    Cookie: 0x00010000ea6f4b8e
    Cookie mask: 0x0000000000000000
    Table ID: 0
    Command: OFPFC_ADD (0)
    Idle timeout: 0
    Hard timeout: 0
    Priority: 40000
    Buffer ID: OFP_NO_BUFFER (4294967295)
    Out port: OFPP_ANY (4294967295)
    Out group: OFPG_ANY (4294967295)
  ▸ Flags: 0x0001
    Importance: 0
  ▾ Match
      Type: OFPMT_OXM (1)
      Length: 10
    ▾ OXM field
        Class: OFPXMC_OPENFLOW_BASIC (0x8000)
        0000 101. = Field: OFPXMT_OFB_ETH_TYPE (5)
        .... ...0 = Has mask: False
        Length: 2
        Value: ARP (0x0806)
    Pad: 000000000000
    Instruction
      Type: OFPIT_CLEAR_ACTIONS (5)
      Length: 8
      Pad: 00000000
  ▾ Instruction
      Type: OFPIT_APPLY_ACTIONS (4)
      Length: 24
      Pad: 00000000
    ▾ Action
        Type: OFPAT_OUTPUT (0)
        Length: 16
        Port: OFPP_CONTROLLER (4294967293)
        Max length: OFPCML_NO_BUFFER (65535)
        Pad: 000000000000
```

```
OpenFlow 1.4
    Version: 1.4 (0x05)
    Type: OFPT_FLOW_MOD (14)
    Length: 96
    Transaction ID: 13
    Cookie: 0x00010000021b41dc
    Cookie mask: 0x0000000000000000
    Table ID: 0
    Command: OFPFC_ADD (0)
    Idle timeout: 0
    Hard timeout: 0
    Priority: 5
    Buffer ID: OFP_NO_BUFFER (4294967295)
    Out port: OFPP_ANY (4294967295)
    Out group: OFPG_ANY (4294967295)
  ▸ Flags: 0x0001
    Importance: 0
  ▾ Match
      Type: OFPMT_OXM (1)
      Length: 10
    ▾ OXM field
        Class: OFPXMC_OPENFLOW_BASIC (0x8000)
        0000 101. = Field: OFPXMT_OFB_ETH_TYPE (5)
        .... ...0 = Has mask: False
        Length: 2
        Value: IPv4 (0x0800)
    Pad: 000000000000
    Instruction
      Type: OFPIT_CLEAR_ACTIONS (5)
      Length: 8
      Pad: 00000000
  ▾ Instruction
      Type: OFPIT_APPLY_ACTIONS (4)
      Length: 24
      Pad: 00000000
    ▾ Action
        Type: OFPAT_OUTPUT (0)
        Length: 16
        Port: OFPP_CONTROLLER (4294967293)
        Max length: OFPCML_NO_BUFFER (65535)
        Pad: 000000000000
```

```
OpenFlow 1.4
    Version: 1.4 (0x05)
    Type: OFPT_FLOW_MOD (14)
    Length: 96
    Transaction ID: 14
    Cookie: 0x000100007a585b6f
    Cookie mask: 0x0000000000000000
    Table ID: 0
    Command: OFPFC_ADD (0)
    Idle timeout: 0
    Hard timeout: 0
    Priority: 40000
    Buffer ID: OFP_NO_BUFFER (4294967295)
    Out port: OFPP_ANY (4294967295)
    Out group: OFPG_ANY (4294967295)
  ▸ Flags: 0x0001
    Importance: 0
  ▾ Match
      Type: OFPMT_OXM (1)
      Length: 10
    ▾ OXM field
        Class: OFPXMC_OPENFLOW_BASIC (0x8000)
        0000 101. = Field: OFPXMT_OFB_ETH_TYPE (5)
        .... ...0 = Has mask: False
        Length: 2
        Value: Unknown (0x8942)
    Pad: 000000000000
  ▸ Instruction
  ▾ Instruction
      Type: OFPIT_APPLY_ACTIONS (4)
      Length: 24
      Pad: 00000000
    ▾ Action
        Type: OFPAT_OUTPUT (0)
        Length: 16
        Port: OFPP_CONTROLLER (4294967293)
        Max length: OFPCML_NO_BUFFER (65535)
        Pad: 000000000000
```

```
OpenFlow 1.4
    Version: 1.4 (0x05)
    Type: OFPT_FLOW_MOD (14)
    Length: 96
    Transaction ID: 15
    Cookie: 0x00010000965555a
    Cookie mask: 0x0000000000000000
    Table ID: 0
    Command: OFPFC_ADD (0)
    Idle timeout: 0
    Hard timeout: 0
    Priority: 40000
    Buffer ID: OFP_NO_BUFFER (4294967295)
    Out port: OFPP_ANY (4294967295)
    Out group: OFPG_ANY (4294967295)
  ▸ Flags: 0x0001
    Importance: 0
  ▾ Match
      Type: OFPMT_OXM (1)
      Length: 10
    ▾ OXM field
        Class: OFPXMC_OPENFLOW_BASIC (0x8000)
        0000 101. = Field: OFPXMT_OFB_ETH_TYPE (5)
        .... ...0 = Has mask: False
        Length: 2
        Value: 802.1 Link Layer Discovery Protocol (LLDP) (0x88cc)
    Pad: 000000000000
    Instruction
      Type: OFPIT_CLEAR_ACTIONS (5)
      Length: 8
      Pad: 00000000
  ▾ Instruction
      Type: OFPIT_APPLY_ACTIONS (4)
      Length: 24
      Pad: 00000000
    ▾ Action
        Type: OFPAT_OUTPUT (0)
        Length: 16
        Port: OFPP_CONTROLLER (4294967293)
        Max length: OFPCML_NO_BUFFER (65535)
        Pad: 000000000000
```

```
OpenFlow 1.4
    Version: 1.4 (0x05)
    Type: OFPT_FLOW_MOD (14)
    Length: 104
    Transaction ID: 16
    Cookie: 0x0023000044a0da45
    Cookie mask: 0x0000000000000000
    Table ID: 0
    Command: OFPFC_ADD (0)
    Idle timeout: 0
    Hard timeout: 0
    Priority: 10
    Buffer ID: OFP_NO_BUFFER (4294967295)
    Out port: OFPP_ANY (4294967295)
    Out group: OFPG_ANY (4294967295)
  ▸ Flags: 0x0001
    Importance: 0
  ▾ Match
      Type: OFPMT_OXM (1)
      Length: 32
    ▾ OXM field
        Class: OFPXMC_OPENFLOW_BASIC (0x8000)
        0000 000. = Field: OFPXMT_OFB_IN_PORT (0)
        .... ...0 = Has mask: False
        Length: 4
        Value: 1
    ▾ OXM field
        Class: OFPXMC_OPENFLOW_BASIC (0x8000)
        0000 011. = Field: OFPXMT_OFB_ETH_DST (3)
        .... ...0 = Has mask: False
        Length: 6
        Value: 1e:83:cd:47:a0:57 (1e:83:cd:47:a0:57)
    ▾ OXM field
        Class: OFPXMC_OPENFLOW_BASIC (0x8000)
        0000 100. = Field: OFPXMT_OFB_ETH_SRC (4)
        .... ...0 = Has mask: False
        Length: 6
        Value: ca:85:40:25:a3:4e (ca:85:40:25:a3:4e)
    Instruction
      Type: OFPIT_APPLY_ACTIONS (4)
      Length: 24
      Pad: 00000000
    ▾ Action
        Type: OFPAT_OUTPUT (0)
        Length: 16
        Port: 2
        Max length: 0
        Pad: 000000000000
```

There are **6** distinct “OFPT_FLOW_MOD” type with command “OFPFC_ADD” headers during the experiment.

| **Match fields** | **actions** | **Timeout values** |
|---|---|---|
| IN_PORT(0) = 1<br>ETH_DST(3) = 1e:83:cd:47:a0:57<br>ETH_RST(4) = ca:85:40:25:a3:4e | Output port = 1 | 0 |
| IN_PORT(0) = 2<br>ETH_DST(3) = de:68:97:6c:7f:9e<br>ETH_RST(4) = da:cc:5d:95:16:9a | Output port = 1 | 0 |
| ETH_TYPE(5) = ARP(0x0806) | Output port = Controller(4294967293) | 0 |
| ETH_TYPE(5) = IPv4(0x0800) | Output port = Controller(4294967293) | 0 |
| ETH_TYPE(5) = Unknow(0x8942) | Output port = Controller(4294967293) | 0 |
| ETH_TYPE(5) = 802.1 Link Layer Discovery Protocol (LLDP) (0x88cc) | Output port = Controller(4294967293) | 0 |

Unknow(0x8942) 對應的是 TRILL（Transparent Interconnection of Lots of Links） 協議

Following the ethernet header, the packet contains a 4-byte magic number, a 2-byte length field, and a 1-byte type field. Additional data are type dependent

# PART2: Install Flow Rules

## 1. Install one flow rule to forward ARP packets


```
▾ 1:
    groupId:        0
    state:          "ADDED"
    life:           60
    liveType:       "UNKNOWN"
    lastSeen:       1726992498217
    packets:        0
    bytes:          0
    id:             "49539596289727779"
    appId:          "org.onosproject.rest"
    priority:       45000
    timeout:        0
    isPermanent:    true
    deviceId:       "of:0000000000000001"
    tableId:        0
    tableName:      "0"
  ▾ treatment:
    ▾ instructions:
      ▾ 0:
          type:     "OUTPUT"
          port:     "ALL"
      deferred:     []
  ▾ selector:
    ▾ criteria:
      ▾ 0:
          type:     "ETH_TYPE"
          ethType:  "0x806"
```


```
mininet> h1 arping h2
ARPING 10.0.0.2
42 bytes from 5a:80:b3:34:53:7d (10.0.0.2): index=0 time=189.800 usec
42 bytes from 5a:80:b3:34:53:7d (10.0.0.2): index=1 time=10.820 usec
42 bytes from 5a:80:b3:34:53:7d (10.0.0.2): index=2 time=6.953 usec
42 bytes from 5a:80:b3:34:53:7d (10.0.0.2): index=3 time=9.288 usec
42 bytes from 5a:80:b3:34:53:7d (10.0.0.2): index=4 time=4.659 usec
42 bytes from 5a:80:b3:34:53:7d (10.0.0.2): index=5 time=8.807 usec
42 bytes from 5a:80:b3:34:53:7d (10.0.0.2): index=6 time=11.642 usec
```

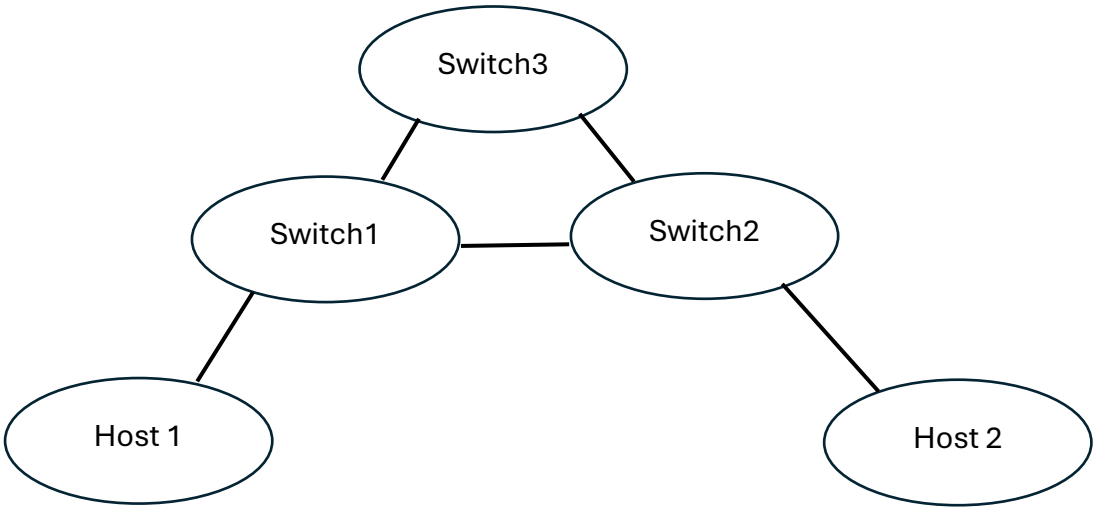
## 2. Install two flow rules to forward IPv4 packets

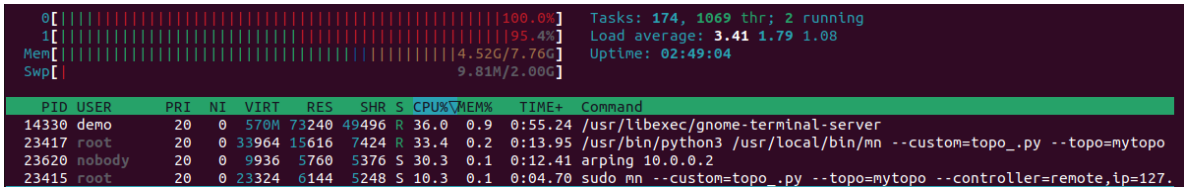

```
▾ 1:
    groupId:        0
    state:          "ADDED"
    life:           24
    liveType:       "UNKNOWN"
    lastSeen:       1726996139175
    packets:        5
    bytes:          490
    id:             "49539597177396634"
    appId:          "org.onosproject.rest"
    priority:       45001
    timeout:        0
    isPermanent:    true
    deviceId:       "of:0000000000000001"
    tableId:        0
    tableName:      "0"
  ▾ treatment:
    ▾ instructions:
      ▾ 0:
          type:     "OUTPUT"
          port:     "1"
      deferred:     []
  ▾ selector:
    ▾ criteria:
      ▾ 0:
          type:     "ETH_TYPE"
          ethType:  "0x800"
      ▾ 1:
          type:     "IPV4_SRC"
          ip:       "10.0.0.2/32"
      ▾ 2:
          type:     "IPV4_DST"
          ip:       "10.0.0.1/32"
```

```
▾ 3:
    groupId:        0
    state:          "ADDED"
    life:           26
    liveType:       "UNKNOWN"
    lastSeen:       1726996139175
    packets:        5
    bytes:          490
    id:             "49539599010961224"
    appId:          "org.onosproject.rest"
    priority:       45001
    timeout:        0
    isPermanent:    true
    deviceId:       "of:0000000000000001"
    tableId:        0
    tableName:      "0"
  ▾ treatment:
    ▾ instructions:
      ▾ 0:
          type:     "OUTPUT"
          port:     "2"
      deferred:     []
  ▾ selector:
    ▾ criteria:
      ▾ 0:
          type:     "ETH_TYPE"
          ethType:  "0x800"
      ▾ 1:
          type:     "IPV4_SRC"
          ip:       "10.0.0.1/32"
      ▾ 2:
          type:     "IPV4_DST"
          ip:       "10.0.0.2/32"
```


```
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.236 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.063 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.091 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.085 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.098 ms
```

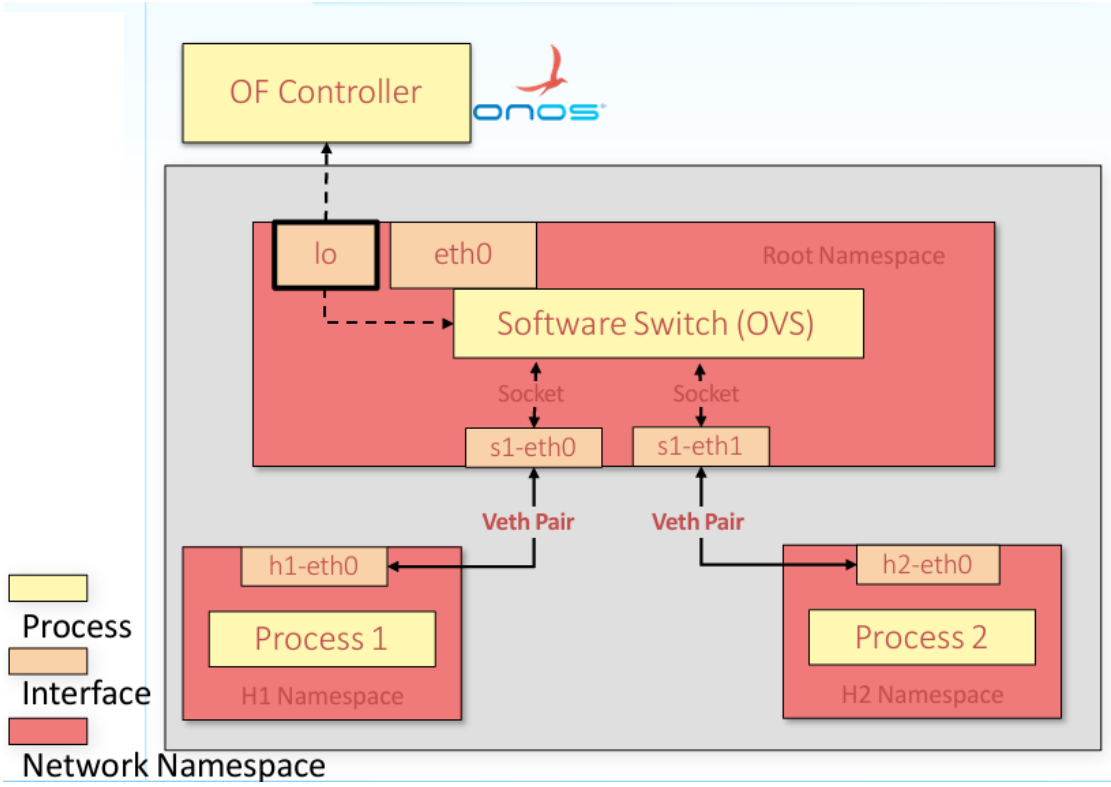
# PART3: Create Topology with Broadcast Storm




我設置的 topology 如上圖所示，對 s1、s2、s3 都設定如同 part2-1 的 flow rule，Forwarding ARP packets to all port。則當 h1 arping h2 時，s1 收到來自 h1 的 broadcast packet 所以複製了一份再傳給 s2、s3，而當 s2、s3 收到的時候也會複製一份往其他地方傳，於是就會一直有 broadcast packet 在 s1、s2、s3 之間互相傳遞，這就會導致封包越來越多，CPU 也需要花費大量的資源在處理這些不斷增生的 broadcast packet。

# Part 4: Trace Reactive Forwarding




**Control plane:**

1. 當 h1 ping h2 時，會發送一個 ICMP (Echo) request 封包，而這個則會進入與 h1 相連的 s1
2. 比對是否有符合的 flow rule，若有則 6.，若無則 3.
3. 因為第一次接收所以沒有符合的 flow rule，則 s1 會將這個封包封裝為 Packet_In 消息，並透過 OVS 傳輸給 ONOS controller
4. ONOS controller 會監聽這些 Packet_In 消息，當收到來自 s1 的封包就開始解析，識別他的來源與目的地
5. ONOS 查詢 topology 資料尋找目的 h2 所在的設備及 port，發現 h2 與 s2 上的 port 相連，於是確認要將封包從 s1 轉發至 s2，並設置 flow rule 於 OVS
6. 發送的封包依照 flow rule 從 s1 經過 s2 最後到達 h2
7. 而當 h2 收到 ping 封包後，則會發送回覆(ICMP Echo Reply)給 h1
8. 因為 flow rule 已經被設置，後續的 ping 則不須再透過 ONOS controller 進行處理

**Data plane:**

- 當 h1 傳輸 ICMP echo request 時候 s1 會收到
- 若沒有相應的 flow rule 則會被傳輸至 ONOS controller
- 若有 flow rule 則依照規則傳輸